

day21笔记

一、RAG介绍

1、什么是RAG？是检索增强生成

2、为什么需要RAG？

- LLM的训练数据是有截止日期的，无法回答最新问题的

示例代码

```
from langchain_community.chat_models.tongyi import ChatTongyi
from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
load_dotenv()

# llm = ChatTongyi(model='qwen3-max')
# result = llm.invoke('你最新的训练数据是哪一年多一天的? ').content
# print(result)

llm = init_chat_model(model='deepseek-chat')
result = llm.invoke('你最新的训练数据是哪一年多一天的? ').content
print(result)
```

输出结果

我的知识截止日期是2025年5月。但具体到“哪一年多一天”这样的细节，由于训练数据涵盖多种来源和不同时间点的信息，无法精确到“某一年比标准年份多一天”这种特殊日期。我的回答基于截至2025年5月的公开知识，如果需要最新动态，建议通过联网搜索功能查询实时信息。

- 幻觉问题，生成一些答案看似合理其实是错误的（因为AI它的循环语料是日期的，你问的问题超出了它的限制，这个时候又不好不给回答，因此会根据以往的经验给你一个答案）
- 缺乏专业知识，LLM可能不了解你的公司、品牌、内部的知识

以上存在的问题，可以通过RAG来解决

3、RAG解决方案

- 用户提问
- 检索相关知识（搭建本地知识库）
- 将检索出来的知识注入给提示词
- 生成精准的答案

二、RAG文档加载

1、安装

```
# pypdf 加载pdf文档
# unstructured 多种文档格式支持
# selenium 处理网络加载
pip install pypdf unstructured selenium
```

2、文本文件加载

```
from langchain_community.chat_models.tongyi import ChatTongyi
from langchain_community.document_loaders import TextLoader
from dotenv import load_dotenv
load_dotenv()

loader = TextLoader('./file/用户问题.txt', encoding='utf-8')

documents = loader.load()

# 注意点: 被加载出来的数据放在一个列表中的、
# page_content 存放文档内容的属性
# metadata 元数据 (表示这个数据的来源是什么)
# print(documents[0].page_content)
# print(documents[0].metadata)

for doc in documents:
    print(doc.page_content)
    print('-' * 30)
    print(doc.metadata)
```

3、CSV文件加载

```
from langchain_community.chat_models.tongyi import ChatTongyi
from langchain_community.document_loaders import CSVLoader
from dotenv import load_dotenv
load_dotenv()

loaders = CSVLoader(
    './file/员工数据表.csv',
    encoding='utf-8',
    source_column='姓名' # 指定以什么作为依据 (某个数据来源于哪里)
)

documents = loaders.load()

for doc in documents:
    print(doc.page_content)
    print('-' * 30)
    print(doc.metadata)
```

4、PDF文件加载

```
from langchain_community.chat_models.tongyi import ChatTongyi
from langchain_community.document_loaders import PyPDFLoader
```

```

from dotenv import load_dotenv
load_dotenv()

loaders = PyPDFLoader('./file/提示词工程.pdf')

documents = loaders.load()

# Ignoring wrong pointing object 45 0 (offset 0) 大家以后看到这个提示，其实不是加载错误，而是表示加载的pdf文档有损坏，或者内容格式存在某些问题
# print(documents)

print(f'共{len(documents)}页')

# 通过索引查看那些页内容
target_pages = documents[0:3]

print(f'查看，共{len(target_pages)}页')

for i, doc in enumerate(target_pages):
    print(f'--- 第{i+1}页 ---')
    print(doc.page_content)

```

5、批量加载文件

```

from langchain_community.document_loaders import DirectoryLoader, TextLoader
from dotenv import load_dotenv
load_dotenv()

# 加载某个文件夹下的所有文件
# *.txt 表示匹配当前目录下所有的.txt文件 例如: a.txt b.txt hello.txt
# **/ 表示递归匹配，意思是不管在那个文件夹下，都可以匹配到
# **/*.txt 整体的意思是可以匹配到./file文件夹下以及所以子文件夹里面的文件。文件后缀做了限定，只能匹配到.txt文件

# loader_cls=TextLoader 使用那个模块加载文件
# show_progress=True 表示显示加载文件的进度
# loader_kwargs={'encoding': 'utf-8'} 设置字符编码
loader = DirectoryLoader(
    path='./file',
    glob='**/*.txt',
    loader_cls=TextLoader,
    show_progress=True,
    loader_kwargs={'encoding': 'utf-8'}
)

documents = loader.load()
print(f'共加载了{len(documents)}个文件')

for document in documents:
    print(document.page_content)
    print('-' * 30)
    print(document.metadata)

```

6、Markdown文件加载

安装

```
pip install markdown
```

示例代码

```
from langchain_community.document_loaders import UnstructuredMarkdownLoader
from dotenv import load_dotenv
load_dotenv()

loaders = UnstructuredMarkdownLoader('./file/day19笔记.md')

documents = loaders.load()

print(documents[0].page_content)
```

7、网页加载

```
import os
# 给langchain查看的，告知身份，一定要放在最前面
os.environ["USER_AGENT"] = 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'

from langchain_community.document_loaders import WebBaseLoader

headers = {'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36'}
# headers, 给WebBaseLoader查看的，如果不添加也会出现这个信息
# loader = WebBaseLoader(
#     'https://www.baidu.com',
#     requests_kwargs={'headers': headers},
# )

# 加载多个网页
loader = WebBaseLoader(
    ['https://www.baidu.com', 'https://www.jd.com'],
    requests_kwargs={'headers': headers},
)

documents = loader.load()

for doc in documents:
    print(doc.page_content.strip())

    print('-' * 50)
```

